COTECNOVA

Diseño e implementación de una base de datos para NorteLeads

Servicio tercerizado de seguimiento comercial para Cartago y el norte del Valle

PROYECTO - PRIMER CORTE (30%)

Antoine Roman

Bases de Datos II

Docente: Jhon James Cano Sánchez

Cartago, Valle del Cauca

23 de septiembre de 2026

Tabla de contenido

Tabla de contenido 2
Resumen 3
1. Introducción 3
1.1 Contexto y problema local . . . 3
1.2 Propuesta y alcance . . . 4
1.3 Objetivos . . . 4
2. Modelo entidad-relación 5
2.1 Diagrama MER . . . 5
2.2 Entidades y función . . . 6
2.3 Relaciones y cardinalidades . . . 6
2.4 Restricciones de integridad . . . 6
3. Esquema relacional 8
3.1 Diagrama relacional . . . 8
3.2 Diccionario de datos . . . 9
3.3 Justificación de tipos . . . 10
4. Procedimientos almacenados 11
4.1 sp_listar_prospectos_negocio . . . 11
4.2 sp_contar_prospectos . . . 11
4.3 sp_registrar_intento . . . 12
5. Triggers 13
5.1 trg_prospecto_auditar_estado . . . 13
6. Datos de prueba y consultas 13
6.1 Cobertura de datos . . . 13
6.2 Consulta con varios INNER JOIN . . . 14
7. Entorno Docker y evidencias 14
7.1 Arquitectura de ejecución . . . 14
7.2 Evidencias de ejecución . . . 15
8. Conclusiones y trabajo futuro 18
8.1 Conclusiones . . . 18
8.2 Trabajo futuro . . . 18
Referencias 19
Anexo A. Estructura del repositorio 20
Anexo B. Secuencia de demostración 20

Resumen

Este informe presenta el diseño y la implementación de **NorteLeads**, una base de datos relacional para un servicio tercerizado de atención y seguimiento comercial dirigido a microempresas de Cartago y el norte del Valle del Cauca. El sistema centraliza negocios clientes, planes, agentes, prospectos, interacciones, citas, suscripciones, pagos y auditoría. La solución se implementa en MariaDB 11.4 dentro de Docker Compose y dispone de phpMyAdmin, una aplicación PHP de verificación, diez tablas con datos de prueba, tres procedimientos almacenados que utilizan parámetros IN, OUT e INOUT, y un trigger básico de auditoría. El diseño busca reducir la pérdida de oportunidades causada por respuestas tardías o seguimientos no registrados. La implementación SQL se limita deliberadamente a los recursos vistos en clase: JOIN directos con alias, procedimientos sencillos y uso de OLD y NEW en triggers.

Palabras clave: MariaDB, Docker, seguimiento comercial, prospectos, procedimientos almacenados, triggers.

1. Introducción

1.1 Contexto y problema local

El tejido empresarial del norte del Valle es amplio, pero está atomizado. La Cámara de Comercio de Cartago reportó 10.796 empresas activas al cierre de 2024 y señaló que aproximadamente el 97 % correspondía a microempresas. En su jurisdicción, Cartago concentraba 5.556 unidades empresariales, seguido por Roldanillo con 1.327 y La Unión con 1.280 (Cámara de Comercio de Cartago, 2025). Esta composición hace pertinente una solución sencilla y de costo controlado que ayude a registrar y seguir oportunidades comerciales sin exigir a cada empresa un equipo interno especializado.

En sectores como salud, educación, inmobiliario, comercio, alojamiento y servicios personales, una parte de la demanda llega por WhatsApp, Instagram, formularios web o llamadas. Cuando la conversación queda únicamente en el teléfono de una persona, se dificulta saber quién respondió, qué prometió, cuándo se debe contactar de nuevo y cuántas consultas terminaron en cita o venta. El problema de información se formula así: ¿cómo centralizar y controlar el ciclo de un prospecto, desde su ingreso hasta su conversión o cierre, para varios negocios clientes y con trazabilidad de las operaciones?

1.2 Propuesta y alcance

NorteLeads es el núcleo de datos para un micro-BPO comercial: el negocio cliente contrata un plan mensual; un agente recibe o realiza contactos; cada prospecto conserva su historial de seguimientos, citas y cambios de estado; y la administración registra suscripciones y pagos. Este primer corte no envía mensajes reales ni almacena datos de clientes verdaderos. Su alcance es académico y se limita al modelo relacional, la lógica de base de datos, los datos ficticios y el entorno reproducible.

1.3 Objetivos

- Diseñar un modelo entidad-relación y un esquema relacional normalizado que representen clientes empresariales, operación comercial, facturación y auditoría.
- Implementar en MariaDB reglas de integridad, datos de prueba, procedimientos almacenados con parámetros IN, OUT e INOUT, y un trigger básico de trazabilidad.
- Desplegar la solución mediante Docker Compose y demostrar su consulta desde PHP, phpMyAdmin y MySQL Workbench mediante un JOIN directo de varias entidades.

2. Modelo entidad-relación

2.1 Diagrama MER

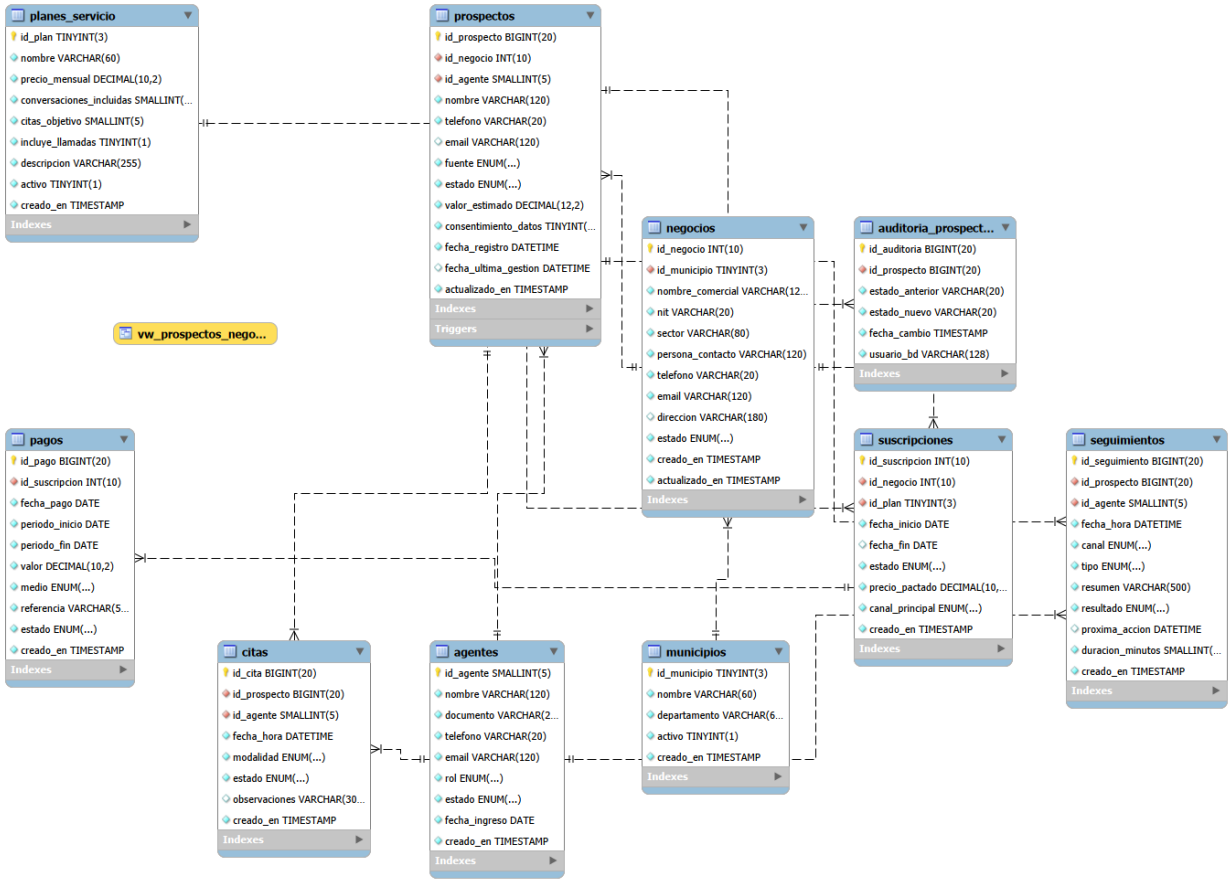


Figura 1. Diagrama entidad-relación de NorteLeads exportado de MySQL Workbench. Nota. Elaboración propia.

El modelo se organiza en cuatro dominios. Los catálogos y recursos comprenden municipios, planes y agentes. La relación contractual está formada por negocios, suscripciones y pagos. La operación comercial se concentra en prospectos, seguimientos y citas. Finalmente, auditoria_prospectos conserva cada transición de estado. Esta separación evita duplicar datos y permite consultar el desempeño por cliente, plan, municipio o agente.

2.2 Entidades y función

Entidad	Descripción	Atributos representativos
municipios	Catálogo geográfico de la zona de atención.	id_municipio, nombre, departamento, activo
planes_servicio	Oferta comercial y capacidad incluida.	id_plan, precio_mensual, conversaciones, citas_objetivo
agentes	Personas que administran o atienden prospectos.	id_agente, documento, email, rol, estado
negocios	Microempresas que contratan el servicio.	id_negocio, nit, sector, contacto, municipio, estado
suscripciones	Vínculo temporal entre negocio y plan.	id_suscripcion, fechas, precio_pactado, canal, estado
prospectos	Interesados gestionados en nombre de cada negocio.	id_prospecto, teléfono, fuente, estado, valor, consentimiento
seguimientos	Interacciones entrantes o salientes.	fecha_hora, canal, resumen, resultado, próxima_accion
citas	Compromisos comerciales derivados del seguimiento.	fecha_hora, modalidad, estado, observaciones
pagos	Recaudo asociado a una suscripción.	periodo, valor, medio, referencia, estado
auditoria_prospectos	Historial de actualizaciones del prospecto.	id_auditoria, id_prospecto, estados, usuario_bd, fecha

2.3 Relaciones y cardinalidades

Relación	Cardinalidad	Tipo	Regla
municipios-negocios	1:N	No identificante	Un negocio pertenece a un municipio.
negocios-suscripciones	1:N	No identificante	Conserva el historial contractual.
planes-suscripciones	1:N	No identificante	Cada suscripción selecciona un plan.
suscripciones-pagos	1:N	No identificante	Un contrato puede recibir varios pagos.
negocios-prospectos	1:N	No identificante	El teléfono es único dentro del negocio.
agentes-prospectos	1:N	No identificante	Cada prospecto tiene responsable.
prospectos-seguimientos	1:N	No identificante	El historial admite múltiples contactos.
prospectos-citas	1:N	No identificante	Puede haber reprogramaciones.
prospectos-auditoría	1:N	No identificante	La auditoría posee su propia PK.

En esta versión todas las relaciones son no identificantes porque cada tabla hija posee una clave primaria propia. Una relación sería identificante si la clave primaria de la tabla padre formara parte de la clave primaria de la hija.

2.4 Restricciones de integridad

- Integridad de entidad: todas las tablas poseen una PK propia; la auditoría usa id_auditoria autoincremental.

- Integridad referencial: las FK usan RESTRICT para catálogos y contratos; los seguimientos, citas y auditoría usan CASCADE al borrar un prospecto.
- Integridad de dominio: ENUM limita estados, canales y resultados; CHECK evita valores negativos y periodos invertidos.
- Integridad de negocio: NIT, emails administrativos y referencias de pago son UNIQUE; el teléfono solo se repite entre negocios diferentes.
- Obligatoriedad: NOT NULL se aplica a identificadores, estados, montos y atributos indispensables; los datos opcionales, como email de prospecto, aceptan NULL.

3. Esquema relacional

3.1 Diagrama relacional

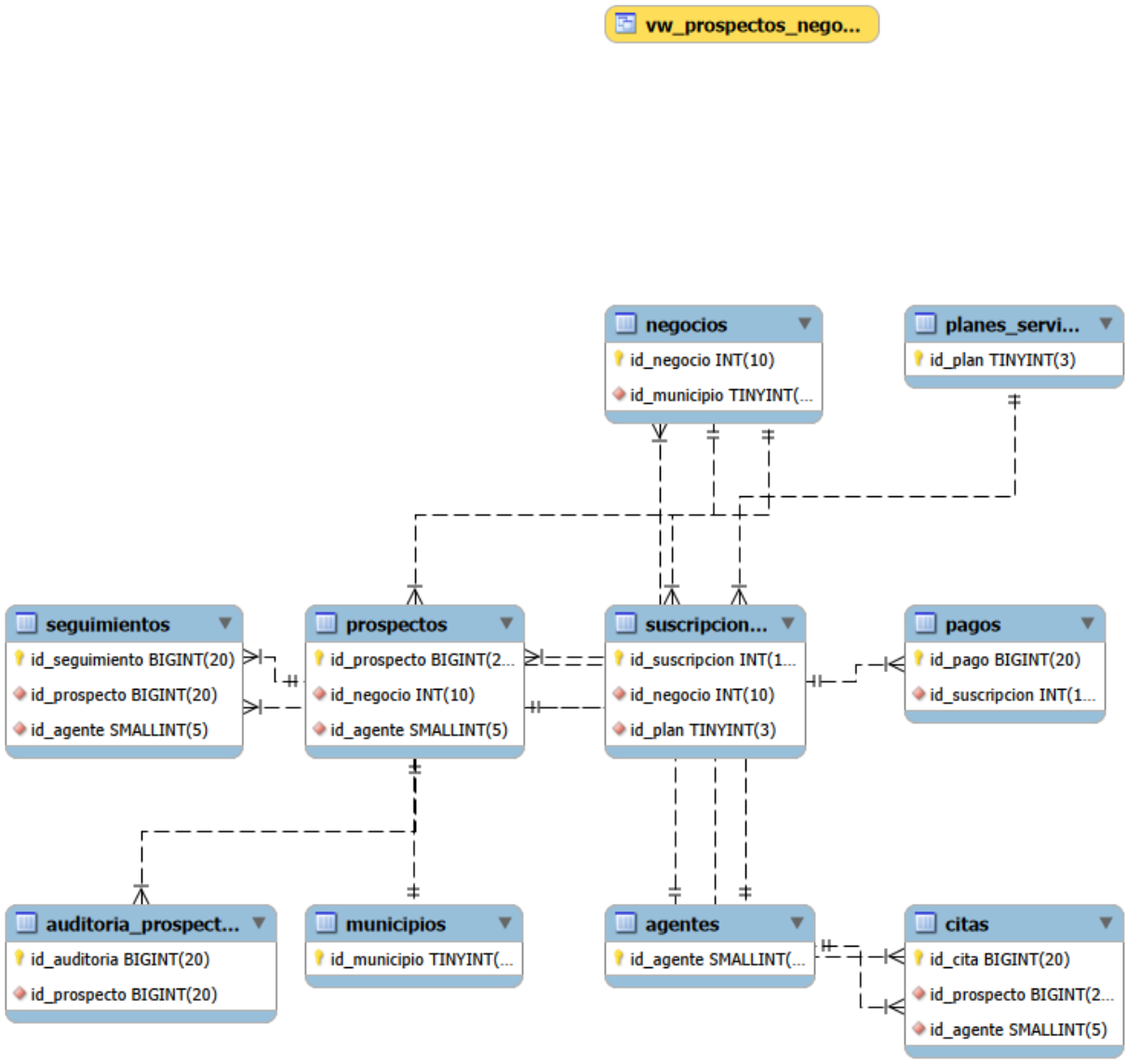


Figura 2. Esquema relacional exportado de MySQL Workbench, con claves primarias y foráneas. Nota. Elaboración propia.

3.2 Diccionario de datos

Tabla	Columnas y tipos	Restricciones principales
municipios	id_municipio TINYINT UNSIGNED nombre VARCHAR(60) departamento VARCHAR(60) activo BOOLEAN creado_en TIMESTAMP	PK id_municipio; UNIQUE nombre+departamento; NOT NULL.
planes_servicio	id_plan TINYINT UNSIGNED nombre VARCHAR(60) precio_mensual DECIMAL(10,2) conversaciones_incluidas SMALLINT citas_objetivo SMALLINT incluye_llamadas BOOLEAN descripcion VARCHAR(255) activo BOOLEAN	PK; UNIQUE nombre; CHECK precio y conversaciones mayores que cero.
agentes	id_agente SMALLINT UNSIGNED nombre VARCHAR(120) documento VARCHAR(20) telefono VARCHAR(20) email VARCHAR(120) rol ENUM estado ENUM fecha_ingreso DATE	PK; UNIQUE documento y email; NOT NULL en datos operativos.
negocios	id_negocio INT UNSIGNED id_municipio TINYINT nombre_comercial VARCHAR(120) nit VARCHAR(20) sector VARCHAR(80) persona_contacto VARCHAR(120) telefono VARCHAR(20) email VARCHAR(120) direccion VARCHAR(180) estado ENUM	PK; FK municipio; UNIQUE NIT y email; dirección admite NULL.
suscripciones	id_suscripcion INT UNSIGNED id_negocio INT id_plan TINYINT fecha_inicio/fecha_fin DATE estado ENUM precio_pactado DECIMAL(10,2) canal_principal ENUM	PK; dos FK; UNIQUE negocio+fecha_inicio; CHECK fechas y precio.
prospectos	id_prospecto BIGINT UNSIGNED id_negocio INT id_agente SMALLINT nombre VARCHAR(120) telefono VARCHAR(20) email VARCHAR(120) fuente/estado ENUM valor_estimado DECIMAL(12,2) consentimiento_datos BOOLEAN fechas DATETIME/TIMESTAMP	PK; dos FK; UNIQUE negocio+teléfono; CHECK valor; índices por estado y fecha.
seguimientos	id_seguimiento BIGINT UNSIGNED id_prospecto BIGINT id_agente SMALLINT fecha_hora DATETIME canal/tipo/resultado ENUM resumen VARCHAR(500) proxima_accion DATETIME duracion_minutos SMALLINT	PK; dos FK; índices por fecha y resultado; próxima acción admite NULL.

Tabla	Columnas y tipos	Restricciones principales
citas	id_cita BIGINT UNSIGNED id_prospecto BIGINT id_agente SMALLINT fecha_hora DATETIME modalidad/estado ENUM observaciones VARCHAR(300)	PK; dos FK; índice compuesto fecha+estado; observaciones admite NULL.
pagos	id_pago BIGINT UNSIGNED id_suscripcion INT fecha_pago DATE periodo_inicio/fin DATE valor DECIMAL(10,2) medio/estado ENUM referencia VARCHAR(50)	PK; FK suscripción; UNIQUE referencia; CHECK periodo y valor; índice fecha+estado.
auditoria_prospectos	id_auditoria BIGINT UNSIGNED id_prospecto BIGINT UNSIGNED estado_anterior/nuevo VARCHAR(20) fecha_cambio TIMESTAMP usuario_bd VARCHAR(128)	PK id_auditoria; FK con CASCADE; índice por fecha.

3.3 Justificación de tipos

Tipo	Uso	Justificación
TINYINT/SMALLINT	Catálogos y agentes	Reduce espacio cuando el dominio esperado es pequeño.
INT/BIGINT UNSIGNED	Negocios y eventos	BIGINT soporta alto volumen histórico; UNSIGNED evita valores negativos.
DECIMAL	Precios y valores	Mantiene exactitud decimal, a diferencia de FLOAT.
VARCHAR	Nombres, teléfonos, resúmenes	Conserva ceros y símbolos del teléfono y limita longitud.
DATE/DATETIME/TIMESTAMP	Periodos y eventos	DATE modela periodos; DATETIME conserva hora; TIMESTAMP automatiza auditoría.
ENUM/BOOLEAN	Estados y banderas	Restringe dominios cortos y evita valores inconsistentes.

4. Procedimientos almacenados

4.1 sp_listar_prospectos_negocio

Propósito. Recibir el identificador de un negocio mediante un parámetro IN y mostrar sus prospectos junto con el agente responsable. El SELECT usa un INNER JOIN directo, un alias y una condición WHERE.

```
DROP PROCEDURE IF EXISTS sp_listar_prospectos_negocio$$
-- Procedimiento 1: recibe un parametro IN y muestra los prospectos del negocio.
CREATE PROCEDURE sp_listar_prospectos_negocio(
    IN p_id_negocio INT UNSIGNED
)
BEGIN
    SELECT
        p.id_prospecto,
        p.nombre AS prospecto,
        p.estado,
        a.nombre AS agente
    FROM prospectos p
    INNER JOIN agentes a
        ON p.id_agente = a.id_agente
    WHERE p.id_negocio = p_id_negocio
    ORDER BY p.nombre;
END$$
```

Llamada de prueba:

```
CALL sp_listar_prospectos_negocio(1);
```

4.2 sp_contar_prospectos

Propósito. Recibir el identificador de un negocio mediante IN, contar sus prospectos y entregar el total mediante OUT. Corresponde al patrón básico SELECT COUNT(*) INTO variable visto en clase.

```
DROP PROCEDURE IF EXISTS sp_contar_prospectos$$
-- Procedimiento 2: recibe un IN y devuelve el total mediante un OUT.
CREATE PROCEDURE sp_contar_prospectos(
    IN p_id_negocio INT UNSIGNED,
    OUT p_total INT
)
BEGIN
    SELECT COUNT(*)
    INTO p_total
    FROM prospectos
    WHERE id_negocio = p_id_negocio;
END$$

SET @total=0;
CALL sp_contar_prospectos(1,@total);
SELECT @total;
```

4.3 sp_registrar_intento

Propósito. Recibir los datos mínimos de un seguimiento, incrementar el número de intento suministrado por el llamador y guardar una fila. Combina parámetros IN con un INOUT sin validaciones ni condiciones avanzadas.

```
DROP PROCEDURE IF EXISTS sp_registrar_intento$$
-- Procedimiento 3: incrementa un INOUT y registra un seguimiento sencillo.
CREATE PROCEDURE sp_registrar_intento(
    IN p_id_prospecto BIGINT UNSIGNED,
    IN p_id_agente SMALLINT UNSIGNED,
    IN p_resumen VARCHAR(500),
    INOUT p_numero_intento INT
)
BEGIN
    SET p_numero_intento = p_numero_intento + 1;

    INSERT INTO seguimientos (
        id_prospecto,
        id_agente,
        canal,
        tipo,
        resumen,
        resultado,
        proxima_accion,
        duracion_minutos
    ) VALUES (
        p_id_prospecto,
        p_id_agente,
        'WHATSAPP',
        'SALIENTE',
        CONCAT('Intento ', p_numero_intento, ': ', p_resumen),
        'PENDIENTE',
        NULL,
        0
    );
END$$

SET @intento=2;
CALL sp_registrar_intento(11,8,'Respondio y solicita catalogo',@intento);
SELECT @intento;
```

5. Triggers

5.1 trg_prospecto_auditar_estado

Se ejecuta **AFTER UPDATE** sobre prospectos. Si el estado cambió, inserta una fila en la tabla de auditoría usando OLD.estado como valor anterior y NEW.estado como valor nuevo. Es el mismo patrón básico de auditoría presentado en la guía de triggers.

```
DROP TRIGGER IF EXISTS trg_prospecto_auditar_estado$$
CREATE TRIGGER trg_prospecto_auditar_estado
AFTER UPDATE ON prospectos
FOR EACH ROW
BEGIN
    IF OLD.estado <> NEW.estado THEN
        INSERT INTO auditoria_prospectos (
            id_prospecto,
            estado_anterior,
            estado_nuevo,
            usuario_bd
        ) VALUES (
            NEW.id_prospecto,
            OLD.estado,
            NEW.estado,
            CURRENT_USER()
        );
    END IF;
END$$
```

Ejemplo de prueba:

```
SELECT id_prospecto, estado FROM prospectos WHERE id_prospecto=12;
UPDATE prospectos SET estado='CITA' WHERE id_prospecto=12;
SELECT * FROM auditoria_prospectos WHERE id_prospecto=12;
```

6. Datos de prueba y consultas

6.1 Cobertura de datos

Tabla	Filas iniciales	Cobertura
municipios	10	Municipios de la jurisdicción
planes_servicio	10	Variantes de oferta
agentes	10	Roles y responsables
negocios	10	Sectores y municipios
suscripciones	10	Un contrato por negocio
prospectos	20	Dos por negocio
seguimientos	20	Resultados y canales
citas	10	Estados y modalidades
pagos	10	Medios y estados
auditoria_prospectos	10	Generadas por trigger

Los nombres, teléfonos, correos, NIT y referencias son ficticios y se crearon únicamente para la demostración académica.

6.2 Consulta con varios INNER JOIN

La consulta relaciona seis tablas: negocios, municipios, prospectos, agentes, suscripciones y planes. Solo utiliza columnas explícitas, alias, INNER JOIN y condiciones ON, exactamente como el ejemplo de múltiples tablas visto en clase. No contiene subconsultas, GROUP BY, SUM, CASE ni tablas derivadas.

```
SELECT
    n.nombre_comercial AS negocio,
    m.nombre AS municipio,
    p.nombre AS prospecto,
    p.estado,
    a.nombre AS agente,
    ps.nombre AS plan,
    s.precio_pactado
FROM negocios n
INNER JOIN municipios m
    ON n.id_municipio = m.id_municipio
INNER JOIN prospectos p
    ON n.id_negocio = p.id_negocio
INNER JOIN agentes a
    ON p.id_agente = a.id_agente
INNER JOIN suscripciones s
    ON n.id_negocio = s.id_negocio
INNER JOIN planes_servicio ps
    ON s.id_plan = ps.id_plan
ORDER BY n.nombre_comercial, p.nombre;
```

La captura del resultado de esta consulta aparece en la figura 6.

7. Entorno Docker y evidencias

7.1 Arquitectura de ejecución

Componente	Imagen/puerto	Función
db	mariadb:11.4 / 3307:3306	Ejecuta esquema, datos, procedimientos, triggers y vista.
phpmyadmin	phpmyadmin:5.2-apache / 8081	Inspección visual y ejecución de SQL.
app	php:8.3-apache / 8080	Comprueba conexión y presenta el listado del JOIN.
MySQL Workbench	127.0.0.1:3307	Cliente de escritorio requerido por la guía.

Docker Compose declara la red interna, un volumen persistente y un healthcheck. Los scripts de sql/init se ejecutan en orden cuando el volumen se crea por primera vez. La aplicación PHP inicia después de que MariaDB informa un estado saludable (Docker, 2026).

```
cp .env.example .env
docker compose up -d --build
docker compose ps
```

7.2 Evidencias de ejecución

Estas capturas fueron proporcionadas por el estudiante. Se incluyen en su proporción original; los archivos PNG completos también están en la carpeta evidencias del proyecto.

7.2.1 Contenedores

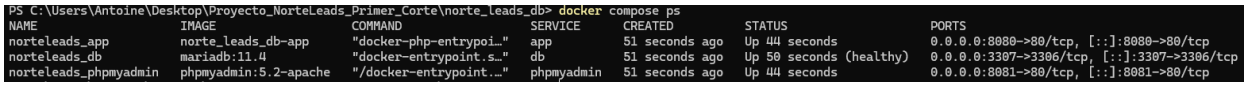


Figura 3. Salida real de docker compose ps: app, db y phpmyadmin en ejecución; puertos 8080, 3307 y 8081.

7.2.2 phpMyAdmin

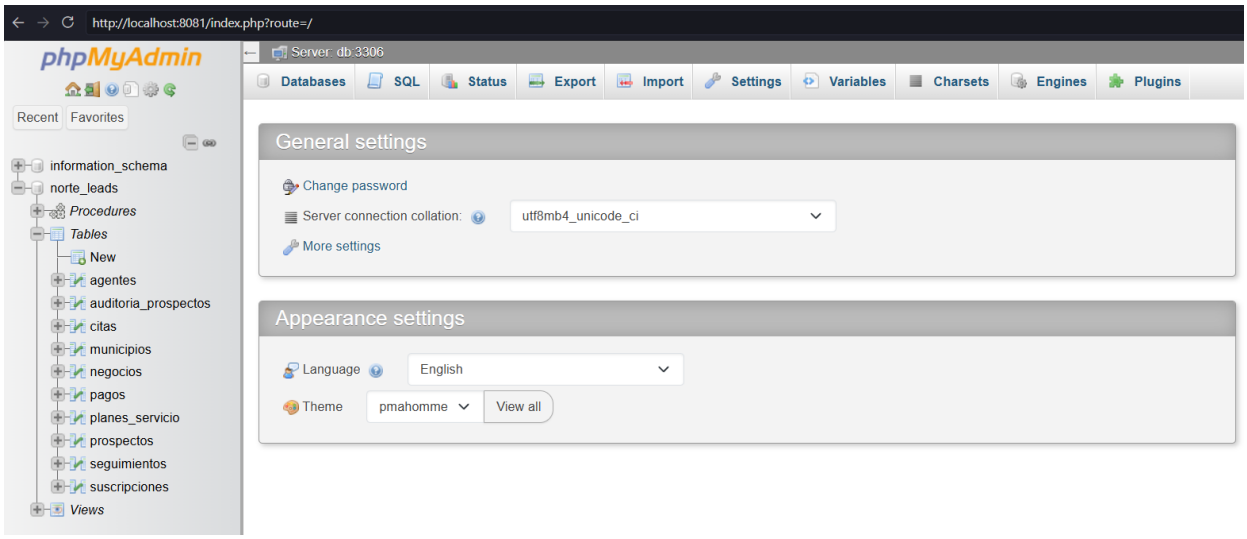


Figura 4. http://localhost:8081 y esquema norte_leads expandido con sus diez tablas visibles en la barra lateral.

7.2.3 MySQL Workbench

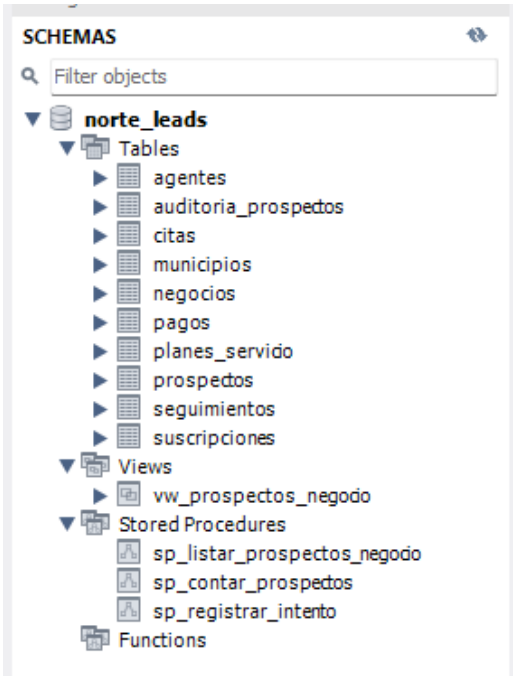


Figura 5. Panel SCHEMAS con las diez tablas, la vista y los tres procedimientos. La captura muestra el esquema abierto, pero no los ajustes de conexión ni la contraseña.

7.2.4 Resultado del JOIN

negocio	municipio	prospecto	estado	agente	plan	precio_pactado
Academia Horizonte	Roldanillo	Samuel Diaz	CONTACTADO	Daniela Velez	Base	750000.00
Academia Horizonte	Roldanillo	Sofia Herrera	PERDIDO	Daniela Velez	Base	750000.00
Agroinsumos La Victoria	La Victoria	Miguel Angel Rojas	NUEVO	Nicolas Arango	Comercio	850000.00
Agroinsumos La Victoria	La Victoria	Valeria Ortiz	NUEVO	Nicolas Arango	Comercio	850000.00
Centro Fitness Zarzal	Zarzal	Isabella Gomez	GANADO	Mariana Quintero	Agenda	900000.00
Centro Fitness Zarzal	Zarzal	Sebastian Cano	PERDIDO	Mariana Quintero	Agenda	900000.00
Clinica Veterinaria Toro	Toro	Emiliano Castro	NUEVO	Andres Patiño	Agenda	900000.00
Clinica Veterinaria Toro	Toro	Gabriela Sanchez	NUEVO	Andres Patiño	Agenda	900000.00
Estetica Natural Alcala	Alcala	Agustin Vargas	NUEVO	Santiago Mora	Agenda	900000.00
Estetica Natural Alcala	Alcala	Luciana Gallego	NUEVO	Santiago Mora	Agenda	900000.00
Hotel Mirador Norte	Anserma...	Martin Osorio	NUEVO	Valentina Rios	Omnicanal	1300000.00
Hotel Mirador Norte	Anserma...	Renata Valencia	NUEVO	Valentina Rios	Omnicanal	1300000.00
Inmobiliaria La Ermita	Cartago	Camila Arias	CITA	Santiago Mora	Crecimie...	950000.00
Inmobiliaria La Ermita	Cartago	Julian Perez	GANADO	Santiago Mora	Crecimie...	950000.00
Muebles del Valle	La Union	Carlos Muñoz	CITA	Juan David Oca...	Comercio	850000.00
Muebles del Valle	La Union	Maria Jose Lopez	CALIFICADO	Juan David Oca...	Comercio	850000.00
OdontoCentro Cartago	Cartago	Ana Beltran	CONTACTADO	Valentina Rios	Agenda	900000.00
OdontoCentro Cartago	Cartago	Luis Cardona	CALIFICADO	Valentina Rios	Agenda	900000.00
Taller Motor Norte	Obando	Mariana Jaramillo	NUEVO	Sara Castaño	Servicios	1050000.00
Taller Motor Norte	Obando	Tomas Escobar	NUEVO	Sara Castaño	Servicios	1050000.00

Figura 6. Resultado real de la consulta de seis tablas: negocio, municipio, prospecto, estado, agente, plan y precio pactado. El código SQL aparece en la sección 6.2; la captura recibida solo contiene la cuadrícula de resultados.

7.2.5 Procedimientos almacenados

Las tres capturas muestran las salidas obtenidas al ejecutar las llamadas del archivo sql/demo/05_demo_calls.sql.

Procedimiento con parámetro IN

id_prospecto	prospecto	estado	agente
1	Ana Beltran	CONTACTADO	Valentina Rios
2	Luis Cardona	CALIFICADO	Valentina Rios

Figura 7. sp_listar_prospectos_negocio(1): dos prospectos del negocio 1.

Procedimiento con parámetro OUT

total_prospectos
2

Figura 8. sp_contar_prospectos(1, @total): devuelve 2.

Procedimiento con parámetro INOUT

id_seguimiento	id_prospecto	id_agente	resumen	resultado
21	11	8	Intento 3: Respondio y solicita catalogo	PENDIENTE
11	11	8	Consulta disponibilidad de fertilizante	PENDIENTE

Figura 9. sp_registrar_intento: se registra el seguimiento 21 con el texto Intento 3. La captura recibida muestra el registro, pero no el SELECT de la variable @intento.

7.2.6 Trigger de auditoría

Antes del UPDATE

id_prospecto	estado
12	NUEVO

Fila registrada por el trigger

id_auditoria	id_prospecto	estado_anterior	estado_nuevo	fecha_cambio
11	12	NUEVO	CITA	2026-09-22 23:36:55

Figura 10. El prospecto 12 estaba en NUEVO. Tras cambiarlo a CITA, el trigger generó la fila de auditoría 11 con estado_anterior NUEVO y estado_nuevo CITA. Las capturas recibidas muestran las cuadrículas de resultados.

8. Conclusiones y trabajo futuro

8.1 Conclusiones

- El modelo traduce una necesidad comercial local en información estructurada: quién consulta, para qué negocio, quién atiende, qué ocurrió y cuál es el siguiente paso.
- La separación entre catálogos, contratos, operación y auditoría reduce duplicidad y permite crecer de diez negocios de prueba a una operación mayor sin rediseñar el núcleo.
- Las restricciones, los procedimientos y el trigger ubican reglas básicas cerca de los datos, de modo que PHP, phpMyAdmin o Workbench obtienen el mismo comportamiento.
- La consulta de varias tablas demuestra cómo reconstruir información útil sin superar el alcance del curso: cada INNER JOIN se apoya en una clave foránea y una condición ON.
- Docker Compose hace reproducible la demostración: un único archivo levanta MariaDB, phpMyAdmin y PHP con el mismo esquema en otro equipo compatible.

8.2 Trabajo futuro

- Agregar autenticación, permisos por rol y separación lógica de los datos de cada negocio.
- Integrar la API oficial de WhatsApp Business sin almacenar credenciales en el repositorio.
- Medir tiempo de primera respuesta, cumplimiento de próxima acción, tasa de asistencia y conversión por fuente.
- Definir políticas de retención, revocación de consentimiento, anonimización y copias de seguridad.
- Crear pruebas automatizadas de procedimientos y un pipeline que levante la base antes de aceptar cambios.

Referencias

Cámara de Comercio de Cartago. (2025). Concepto económico 2024. https://www.camaracartago.org/wp-content/uploads/2025/03/CONCEPTO-ECONOMICO-2024.pdf

Docker, Inc. (2026). Docker Compose. https://docs.docker.com/compose/

MariaDB Foundation. (2026a). CREATE PROCEDURE. https://mariadb.com/docs/server/server-usage/stored-routines/stored-procedures/create-procedure

MariaDB Foundation. (2026b). CREATE TRIGGER. https://mariadb.com/docs/server/server-usage/triggers-events/triggers/create-trigger

Oracle. (2026). MySQL Workbench manual: Connections in MySQL Workbench. https://dev.mysql.com/doc/workbench/en/wb-mysql-connections.html

Anexo A. Estructura del repositorio

```
norte_leads_db/
├── app/                # PHP y Dockerfile
├── docs/               # Informe, diagramas y guion
├── evidencias/         # Capturas reales del estudiante
├── sql/
│   ├── init/           # Esquema, rutinas, triggers, datos y vista
│   └── demo/           # CALL, JOIN y prueba de trigger
├── tools/              # Validación y generación del informe
├── .env.example
├── docker-compose.yml
└── README.md
```

Anexo B. Secuencia de demostración

Minuto	Acción	Evidencia
0-3	Explicar problema, propuesta y objetivos.	Portada e introducción.
3-8	Presentar MER, relaciones e integridad.	Figuras 1 y 2.
8-13	Mostrar Docker, phpMyAdmin, Workbench y JOIN.	Figuras 3 a 6.
13-18	Ejecutar tres procedimientos y trigger.	Figuras 7 a 10.
18-20	Conclusiones y trabajo futuro.	Sección 8.

> Nota final. Antes de entregar, confirmar el nombre del estudiante y publicar el repositorio como público. El archivo docs/checklist_entrega.md reúne todas las verificaciones.